

LAB TEST: ONE-DIMENSIONAL ARRAY USING FUNCTIONS

Duration: 2 Hours

Exercise 1: Array Manipulation – Even and Odd Numbers

Write a C program that performs operations on a one-dimensional array of integers.

Requirements:

- Create a function `arrayOddEven()`
- Declare a 1D integer array of size **12**.
- Allow the user to input 12 integers into the array.
- Count and display the number of even and odd numbers in the array.
- Store all even numbers into a separate array called `even[]`.
- Store all odd numbers into another array called `odd[]`.
- Display both the `even[]` and `odd[]` arrays.

Sample output

```
Number of even elements: 7
Number of odd elements: 5
Even numbers: 2 4 8 10 14 16 20
Odd numbers : 3 5 7 9 11
```

Exercise 2: Student Marks Analysis Using Functions

Write a C program to analyze student marks using a **one-dimensional array and functions**.

Requirements:

- Declare a one-dimensional integer array of size **10** in the `main()` function.
- Write a function `inputMarks()` to read marks for 10 students.
- Write a function `displayMarks()` to display all the marks.
- Write a function `findHighestLowest()` to determine and display:
 - The **highest mark**
 - The **lowest mark**
- Write a function `calculateAverage()` to calculate and return the **average mark**.
- Display the average mark in `main()` (formatted to 2 decimal places).

Sample Output:

```
Enter 10 students mark
Enter mark : 76
Enter mark : 45
Enter mark : 63
Enter mark : 66
Enter mark : 89
Enter mark : 28
Enter mark : 94
Enter mark : 53
Enter mark : 58
Enter mark : 49

You enter 10 students mark as below
76 45 63 66 89 28 94 53 58 49

Highest and lowest marks as below
Highest: 94
Lowest : 28

Average of 10 marks as below
Average: 62.10
```

Exercise 3: Even and Odd Array Processing Using Functions

Write a C program that processes a one-dimensional array using **multiple functions**.

Requirements:

- Declare an integer array of size **12** in `main()`.
- Write a function `readArray()` to input the array elements.
- Write a function `countEvenOdd()` to count the number of even and odd values.
- Write a function `separateEvenOdd()` to store:
 - Even numbers into array `even[]`
 - Odd numbers into array `odd[]`
- Write a function `displayArray()` to display the contents of an array.
- Display the total number of even and odd elements.

Sample output

```
Enter element 1: 45
Enter element 2: 67
Enter element 3: 43
Enter element 4: 68
Enter element 5: 88
Enter element 6: 23
Enter element 7: 43
Enter element 8: 56
Enter element 9: 98
Enter element 10: 52
Enter element 11: 49
Enter element 12: 99

Even numbers (5): 68 88 56 98 52
Odd numbers (7): 45 67 43 23 43 49 99
```

Exercise 4: Search and Reverse Array Using Functions

Develop a C program using a one-dimensional array and **functions** to perform searching and reversing operations.

Requirements:

- Declare an integer array of size **15** in `main()`.
- Write a function `inputArray()` to read the array elements.
- Write a function `searchElement()` that searches for a given number and returns its position (or `-1` if not found).
- Write a function `reverseArray()` to reverse the elements of the array.
- Write a function `displayArray()` to display array elements.
- Display the array **before** and **after** reversing.

Sample Output

```
Enter element 1: 5
Enter element 2: 8
Enter element 3: 12
Enter element 4: 3
Enter element 5: 9
Enter element 6: 15
Enter element 7: 7
Enter element 8: 20
Enter element 9: 4
Enter element 10: 6
Enter element 11: 10
Enter element 12: 18
Enter element 13: 2
Enter element 14: 14
Enter element 15: 11

Enter number to search: 9

Array before reverse:
5 8 12 3 9 15 7 20 4 6 10 18 2 14 11

Number found at position 5

Array after reverse:
11 14 2 18 10 6 4 20 7 15 9 3 12 8 5
```